

VehicleOrderBatch

global class VehicleOrderBatch implements Database.Batchable<sObject>{

global Database.QueryLocator start(Database.BatchableContext bc) {

return Database.getQueryLocator([

SELECT Id, Status__c, Vehicle__c FROM Vehicle_Order__c WHERE Status__c = 'Pending'

]);

}

global void execute(Database.BatchableContext bc, List<Vehicle_Order__c> orderList) {

Set<Id> vehicleIds = new Set<Id>();

for (Vehicle_Order__c order : orderList) {

if (order.Vehicle__c != null) {

vehicleIds.add(order.Vehicle__c);

}

}

if (!vehicleIds.isEmpty()) {

Map<Id, Vehicle__c> vehicleStockMap = new Map<Id, Vehicle__c>([

SELECT Id, Stock_Quantity__c FROM Vehicle__c WHERE Id IN :vehicleIds]

);

List<Vehicle_Order__c> ordersToUpdate = new List<Vehicle_Order__c>();

List<Vehicle__c> vehiclesToUpdate = new List<Vehicle__c>();

for (Vehicle_Order__c order : orderList) {

Vehicle__c vehicle = vehicleStockMap.get(order.Vehicle__c);

if (vehicle != null && vehicle.Stock_Quantity__c > 0) {

order.Status__c = 'Confirmed';

vehicle.Stock_Quantity__c -= 1;

ordersToUpdate.add(order);

vehiclesToUpdate.add(vehicle);

}

}

```
        if (!ordersToUpdate.isEmpty()) update ordersToUpdate;
        if (!vehiclesToUpdate.isEmpty()) update vehiclesToUpdate;
    }
}

global void finish(Database.BatchableContext bc) {
    System.debug('Vehicle order batch job completed.');
```



```
    }
}
```